

Performance

A hands-on course in building agents by building their harness

Lesson 9

A run that finishes can still cost too much

- Lesson 8 made the agent keep going when something fails
- But ten minutes and ten dollars is a run you think twice about starting
- Over a long task two things add up: **the time you wait** and **the tokens you pay for**

Time and tokens come from the same place

Primitive	Where the time and tokens go
Context	what the request holds, resent on every pass
Model interface	which model a kind of request goes to
Output	waiting on commands to finish

Streaming isn't new: `call()` has streamed every response since Lesson 1.

The cost of a task is everything resent

- Every pass of the loop sends the **whole** request again: tools, instructions, every message
- Twenty steps means reading the same opening twenty times
- **Prompt caching:** the request is a prefix, read in a fixed order
- Mark where the remembering ends with `cache_control`; a call that starts with the same tokens only reads the new end
- A cache read costs a tenth of normal input; a write, a quarter more; it lasts five minutes since last use

A cache has three catches

- **The match has to be exact.** One changed character early and everything after it is new. That's why Lesson 4's prompt has the date, not the time
- **There's a minimum.** 512 to 4,096 tokens depending on the model; quark's system prompt is about nine thousand tokens
- **It belongs to one model.** Switch models and you start again

The system prompt has carried a mark since Lesson 4. New here: a second mark on the **last message**.

Send less, and wait together

- **Trim:** cap what one tool result may add, keeping the start and the end
- **Small jobs to the small model:** compaction always goes to a faster, cheaper one
- **Run together:** several tool requests in one response run at the same time
- Results still go back in order, each with its id; the guard's questions come first, one at a time
- One ESC has to stop all of them, not just the first

Model interface: a list of models (abridged)

```
MODELS, FAST = ["claude-sonnet-5-5", "claude-opus-5-5"], ["claude-haiku-4-5"]  
class Down(Exception): pass  
def call(each=lambda event: None, models=MODELS, **request):  
    for model in models:
```

Only compaction asks for the small model:

```
summary = call(models=FAST, max_tokens=2048, system=system(), messages=keep + [...])
```

Context: trim, and cache the end (abridged)

```
MAX_RESULT = 20_000
def trim(text):
    if len(text) <= MAX_RESULT: return text
    return text[:MAX_RESULT // 2] + f"\n[... {len(text) - MAX_RESULT} characters cut ...]\n" + ...

def cached(working_memory):
    last = working_memory[-1]
    blocks = [{"type": "text", "text": last["content"]} if isinstance(...) else list(last["content"])
    blocks[-1] = {"**blocks[-1], "cache_control": {"type": "ephemeral"}}
    return working_memory[:-1] + [{"role": last["role"], "content": blocks}]
```

```
response = call(unless_esc, max_tokens=16384, ..., messages=cached(working_memory))
```

Output: one command becomes a function (abridged)

```
def execute(cmd):  
    if ESC.is_set(): return "[your doing never reached the world]"  
    start = time.time()  
    doing = subprocess.Popen(["docker", "exec", box, "timeout", ..., cmd], stdout=subprocess.PIPE, ...)  
    while True:  
        try: done = subprocess.CompletedProcess(doing.args, 0, doing.communicate(timeout=0.1)[0]); break  
        except subprocess.TimeoutExpired:  
            if ESC.is_set(): subprocess.run(["docker", "exec", box, "sh", "-c", "kill -9 -1"], ...)  
    done.returncode = doing.returncode  
    if ESC.is_set(): done.stdout += "\n[your doing stopped before done]"  
    elif done.returncode == 137: done.stdout += f"\n(killed: ...)"  
    trace(event="tool", cmd=cmd, seconds=..., exit=done.returncode, chars=len(done.stdout))  
    return trim(done.stdout) or f"(exit {done.returncode})"
```

Decide each request first, in order (abridged)

```
refused, pending = {}, {}  
for block in output:  
    if block.type == "tool_use":  
        cmd = block.input.get("cmd")  
        print(f"$ {cmd}")  
        if not cmd or (response.stop_reason == "max_tokens" and block is output[-1]):  
            refused[block.id] = "[your doing was cut off before it was fully formed – ...]"  
        elif (no := guard(cmd)):  
            trace(event="refused", cmd=cmd, why=no)  
            refused[block.id] = no  
        else:  
            pending[block.id] = cmd
```

Then run everything allowed, at once (abridged)

```
with listening(), ThreadPoolExecutor() as pool:
    outputs = dict(zip(pending, pool.map(execute, pending.values())))

input = []
for block in output:
    if block.type == "tool_use":
        text = refused.get(block.id) or outputs[block.id]
        print(text if block.id in outputs or text.startswith("[") else f"[{text}]")
        input.append({"type": "tool_result", "tool_use_id": block.id, "content": text, ...})
```

One prompt line changes: *Commands that don't depend on each other can go in the same response: they run at the same time.*

Three seconds of waiting, not nine

Three `sleep 3` commands that don't depend on each other. This lesson's `quark.py`:

```
$ sleep 3; echo lint ok
allow `sleep 3; echo lint ok`? [y/N] $ sleep 3; echo types ok
allow `sleep 3; echo types ok`? [y/N] $ sleep 3; echo tests ok
allow `sleep 3; echo tests ok`? [y/N] lint ok

types ok

tests ok

Lint, types, and tests all passed.
[wall: 8.0s]
```

Lesson 8's `quark.py` ran them one after another: `[wall: 14.4s]` .

The trace shows it, and the cache too

Lesson 8:

```
{"event": "model", "seconds": 1.97, "input_tokens": 145, "cache_read": 0, "cache_write": 8709}  
{"event": "tool", "seconds": 3.11, "cmd": "sleep 3; echo lint ok"}  
{"event": "tool", "seconds": 3.08, "cmd": "sleep 3; echo types ok"}  
{"event": "tool", "seconds": 3.08, "cmd": "sleep 3; echo tests ok"}  
{"event": "model", "seconds": 1.22, "input_tokens": 407, "cache_read": 8709, "cache_write": 0}
```

This lesson:

```
{"event": "model", "seconds": 1.84, "input_tokens": 4, "cache_read": 0, "cache_write": 9262}  
{"event": "tool", "seconds": 3.11, "cmd": "sleep 3; echo types ok"}  
{"event": "tool", "seconds": 3.11, "cmd": "sleep 3; echo tests ok"}  
{"event": "tool", "seconds": 3.11, "cmd": "sleep 3; echo lint ok"}  
{"event": "model", "seconds": 1.23, "input_tokens": 2, "cache_read": 9262, "cache_write": 264}
```

Pay full price for what's new, and keep it short

A three-step task, one command per step:

```
{"seconds":1.52,"input_tokens":4,"output_tokens":67,"cache_read":0,"cache_write":9250}  
{"seconds":1.42,"input_tokens":2,"output_tokens":75,"cache_read":9250,"cache_write":153}  
{"seconds":1.77,"input_tokens":2,"output_tokens":111,"cache_read":9403,"cache_write":125}
```

`cat docs/the-model.md`, a file over 25,000 characters:

```
{"event":"model","seconds":1.62,"input_tokens":4,"cache_read":0,"cache_write":9220}  
{"event":"tool","seconds":0.1,"chars":25435}  
{"event":"model","seconds":2.24,"input_tokens":2,"cache_read":9220,"cache_write":6533}
```

One ESC stops both commands (shortened)

```
$ for i in 1 2 3 4 5 6 7 8 9; do echo a$i; sleep 1; done
allow `for i in 1 2 3 4 5 6 7 8 9; do echo a$i; sleep 1; done`? [y/N] y
$ for i in 1 2 3 4 5 6 7 8 9; do echo b$i; sleep 1; done
allow `for i in 1 2 3 4 5 6 7 8 9; do echo b$i; sleep 1; done`? [y/N] y
a1
...
a5

[your doing stopped before done]
b1
...
b5

[your doing stopped before done]
```

Then the model: *"You interrupted me with ESC, so I stopped both commands."*

What else performance can be

- **What gets cached:** up to four marks; a one-hour lifetime for agents that wait on people
- **What goes in:** clear old tool results; let the model fetch what it needs
- **How much comes back:** "be brief" does more than `max_tokens` ; lower thinking effort
- **How many at once:** cap parallel tools; the batch interface at about half the price
- **As a product:** the API's own settings; [LiteLLM](#) and [OpenRouter](#) choose models per request

performance.py: a small model routes

Lesson 3's loop, async, measuring itself. One call to a small model routes each task:

```
ROUTER = "claude-haiku-4-5"
TIERS = {                                     # tier -> (model, effort)
    "quick": ("claude-haiku-4-5", None),
    "standard": ("claude-sonnet-5-5", "medium"),
    "deep": ("claude-opus-5-5", "high"),
}
```

- Every call prints time to first output and how input splits: new, from cache, written
- Its prompt holds the project's README and docs/, so there's something to cache

Warm cache, cheaper bill; a guess, wrong

```
[done in 2 steps, 1.7s, 50% of input tokens read from the cache]  
[done in 2 steps, 1.9s, 100% of input tokens read from the cache]
```

Same task twice: 50% of input from the cache cold, 100% warm. The saving was in the bill, not the clock.

```
[routed to deep: claude-opus-5-5]  
[3 commands: 3.0s together, 9.0s one after another]
```

Three `echo` s sent to the deep tier: more model than they need.

The rule

Don't resend what hasn't changed, don't send what isn't needed, and don't wait for things one at a time that could be waiting together.

Performance changes how, never what

- **Control flow:** the same loop, the same stops, the same number of calls
- **Input:** the person's task is read the way it always was
- **Context** still decides what the request holds, only marked for reuse
- **Model interface** still sends a request and gets a response, only to a smaller model for a summary
- **Output** still runs what the response asks for, only together, and stopped together

Faster and cheaper, but as good?

- A trimmed result might have cut the line that mattered
- A summary from the small model might have lost a fact
- The router might send a hard task to a weak model
- Parallel commands can quietly depend on each other
- The trace shows a faster, cheaper run, and not whether it did the job

Next: Evaluation

Lesson 10